

Outline & LLD – Automasi Network Operations PT Agiva Indonesia

Ringkasan

Sistem automasi network operations untuk PT Agiva Indonesia mencakup tiga modul: dashboard monitoring real-time, bot alerting Telegram, dan backup konfigurasi terjadwal, seluruhnya berbasis Python dan Telegram Bot API.

Judul laporan acuan: Rancang Bangun Sistem Automasi Network Operations Berbasis Python dan Telegram Bot API: Dashboard Monitoring, Sistem Alerting, dan Backup Konfigurasi Jaringan pada PT Agiva Indonesia.

Dokumen ini memuat outline laporan kerja praktik (Bagian 1) dan Low-Level Design sistem (Bagian 2).

Bagian 1 – Outline Laporan Kerja Praktik

Struktur mengikuti format standar laporan kerja praktik Politeknik: enam bab dari pendahuluan sampai penutup.

1. BAB I Pendahuluan

- o Latar Belakang
- o Rumusan Masalah
- o Tujuan (Umum & Khusus)
- o Manfaat
- o Ruang Lingkup / Batasan Masalah
- o Metodologi Pelaksanaan
- o Sistematika Penulisan

2. BAB II Gambaran Umum Perusahaan

- o Profil PT Agiva Indonesia
- o Visi dan Misi
- o Struktur Organisasi
- o Bidang Layanan (Network Connectivity, Security, Data Center, Next-Gen Network)
- o Deskripsi Divisi Tempaat Praktik (Network Engineering/NOC)

3. BAB III Landasan Teori

- o Network Automation
- o Python untuk Network Automation
- o Telegram Bot API
- o Network Monitoring & Alerting System
- o Configuration Backup & Version Control
- o Perangkat Jaringan (MikroTik RouterOS, Cisco IOS)

4. BAB IV Analisis dan Perancangan Sistem

- o Analisis Kebutuhan (Functional & Non-Functional)
- o Analisis Masalah Operasional NOC
- o Arsitektur Sistem (ringkas, detail lengkap di Bagian 2)
- o Alur Kerja Tiap Modul
- o Desain Antarmuka Dashboard

5. BAB V Implementasi dan Pengujian

- o Implementasi Data Collector
- o Implementasi Dashboard Monitoring
- o Implementasi Bot Alerting Telegram
- o Implementasi Script Backup Konfigurasi
- o Integrasi Antarmodul
- o Pengujian (Unit, Fungsional, UAT)
- o Hasil dan Pembahasan

6. BAB VI Penutup

- o Kesimpulan
- o Saran

Dilengkapi Daftar Pustaka dan Lampiran (cuplikan source code, dokumentasi konfigurasi, screenshot dashboard/bot, log harian kegiatan magang).

Bagian 2 – Low-Level Design (LLD)

Arsitektur Sistem

Enam komponen saling terhubung lewat satu data store bersama, sehingga tiap modul bisa berjalan atau gagal independen tanpa menghentikan modul lain.

flowchart TD

```
A[Perangkat Jaringan] -->|SSH/API| B[Data Collector]
B --> C[(Data Store)]
C --> D[Dashboard Monitoring]
C --> E[Rule Engine]
E --> F[Bot Telegram]
G[Scheduler Backup] --> H[Modul Backup Config]
H --> I[(Storage Config)]
H --> F
```

Komponen	Fungsi	Teknologi
Data Collector	Polling status & metrik perangkat	Python, Netmiko, RouterOS-API
Data Store	Simpan status, histori, log alert	SQLite / PostgreSQL
Dashboard Monitoring	Visualisasi status real-time	Flask/Streamlit, Chart.js
Rule Engine	Evaluasi anomali, trigger alert	Python
Bot Telegram	Kirim notifikasi, terima command	python-telegram-bot
Modul Backup	Ambil & simpan config perangkat	Python, Netmiko
Scheduler	Jalankan job polling & backup periodik	APScheduler / cron

Tiga prinsip desain: modular (tiap modul berdiri sendiri), idempotent (polling dan backup aman diulang), fail-safe (kegagalan satu modul tidak menghentikan modul lain).

Modul Data Collector dan Skema Basis Data

Data collector melakukan polling status dan metrik perangkat secara periodik lewat RouterOS-API (MikroTik) atau SSH via Netmiko (Cisco), dijadwalkan tiap 1–5 menit dengan APScheduler.

Struktur modul (collector.py):

1. `load_inventory()` – baca daftar perangkat dari `devices.yaml`
2. `poll_device(device)` – cek reachability lalu ambil metrik (status interface, CPU, memory, uptime)
3. `save_metrics(device_id, metrics)` – simpan ke data store
4. `run_polling_cycle()` – loop semua device, dipanggil scheduler

Skema basis data:

Tabel	Kolom Kunci	Keterangan
devices	device_id (PK), name, ip_address, vendor, site, credential_ref	Inventaris perangkat
device_status_logs	log_id (PK), device_id (FK), timestamp, status, cpu_usage, mem_usage, interface_status	Histori metrik hasil polling
alerts	alert_id (PK), device_id (FK), type, triggered_at, acknowledged, resolved_at	Log kejadian anomali
backup_history	backup_id (PK), device_id (FK), timestamp, file_path, status	Log hasil backup config

credential_ref menyimpan referensi ke secret store, bukan password perangkat secara langsung.

Modul Dashboard Monitoring dan Bot Alerting Telegram

Dashboard dibangun dengan Flask (atau Streamlit untuk prototipe cepat) plus Chart.js untuk grafik histori; bot alerting memakai python-telegram-bot.

Halaman/route dashboard:

Route	Fungsi
/	Ringkasan status seluruh device (up/down, alert aktif)
/device/<id>	Detail histori CPU/memory/traffic satu device
/alerts	Daftar alert aktif dan riwayat
/api/status	Endpoint JSON untuk data real-time (dipolling frontend)

Rule engine trigger alert: device tidak merespons polling berturut-turut, penggunaan CPU/memory melewati threshold (mis. 85%) selama beberapa menit, atau interface utama down.

Command bot untuk tim NOC:

Command	Fungsi
<code>/status</code>	Ringkasan status semua device saat ini
<code>/device <nama></code>	Status detail satu device
<code>/ack <alert_id></code>	Acknowledge alert
<code>/backup <nama></code>	Trigger backup manual satu device

Anti-spam: alert yang sama tidak dikirim ulang selama belum resolved, dengan cooldown per jenis alert per device.

Modul Backup Konfigurasi Jaringan

Config diambil terjadwal harian (dini hari) lewat perintah `export` (MikroTik) atau `show running-config` via Netmiko (Cisco), dengan opsi trigger manual lewat command bot `/backup`.

File disimpan dengan struktur `/backups/{site}/{device_name}/{YYYY-MM-DD}_config.txt`; folder backup opsional diinisialisasi sebagai Git repository agar tiap perubahan config tersimpan sebagai commit.

Retention policy menyimpan backup 30 hari terakhir per device, dengan penghapusan otomatis untuk file yang lebih lama. Ringkasan hasil backup (sukses/gagal) dikirim ke Telegram tiap siklus selesai; kegagalan per device juga dikirim sebagai alert terpisah.

Alur Data End-to-End

```
sequenceDiagram
    participant S as Scheduler
    participant C as Collector
    participant D as Data Store
    participant R as Rule Engine
    participant B as Bot Telegram
    participant N as Tim NOC
    S->>C: Trigger polling
    C->>D: Simpan metrik
    D->>R: Evaluasi threshold
    R->>B: Kirim alert jika anomali
    B->>N: Notifikasi Telegram
    N->>B: /ack alert_id
```

Alur backup berjalan terpisah: scheduler memicu job harian, modul backup SSH/API ke tiap device, config disimpan ke folder terstruktur, hasil dicatat ke `backup_history`, ringkasan dikirim ke Telegram.

Keamanan, Error Handling, dan Logging

Kredensial perangkat dan token bot disimpan sebagai environment variable lewat `python-dotenv`, tidak pernah hardcode di source code. Akun collector idealnya read-only; command bot yang mengubah state (`/ack`, `/backup`) dibatasi lewat whitelist chat ID tim NOC.

Setiap koneksi ke device dibungkus try/except dengan retry terbatas (2x) dan timeout eksplisit, sehingga satu device bermasalah tidak memblokir siklus polling device lain. Kegagalan pengiriman pesan Telegram di-log dan di-retry, bukan menghentikan proses utama.

Logging memakai modul logging bawaan Python (level INFO/WARNING/ERROR) dengan rotasi file lewat RotatingFileHandler; log ERROR kritis opsional turut dikirim sebagai ringkasan ke Telegram.

Deployment Architecture dan Tech Stack

Seluruh modul berjalan di satu server/VM internal Agiva, atau dipisah per container Docker untuk isolasi lebih baik. Scheduler mengatur siklus polling dan backup otomatis; dashboard di-expose terbatas ke jaringan internal, diakses tim NOC lewat browser.

Kategori	Teknologi
Bahasa pemrograman	Python 3.x
Automasi perangkat	Netmiko, RouterOS-API
Web framework dashboard	Flask / Streamlit
Visualisasi	Chart.js / Plotly
Bot	python-telegram-bot
Basis data	SQLite (prototipe) / PostgreSQL (produksi)
Scheduler	APScheduler / cron
Manajemen secret	python-dotenv
Version control config (opsional)	Git

Nama fungsi, skema tabel, dan pilihan library final dapat disesuaikan dengan kondisi aktual di lapangan: jumlah device, akses ke server produksi Agiva, dan preferensi pembimbing.